Arquitectura de los dos ejecutables y contrato ABI

Entrada binaria
N int32 LE + N float32 LE

lectura

driver.c
Validar archivo y AVX2; reservar entrada/salida a 32 bytes
Medir llamadas, obtener sigma con sqrtf, escribir y liberar

ejecutable escalar

ejecutable AVX2

libstats_scalar.asm
Un float por iteración
sum_array / compute_stats / normalize_array

libstats_vector.asm
Ocho floats por grupo + cola escalar
sum_array / compute_stats / normalize_array

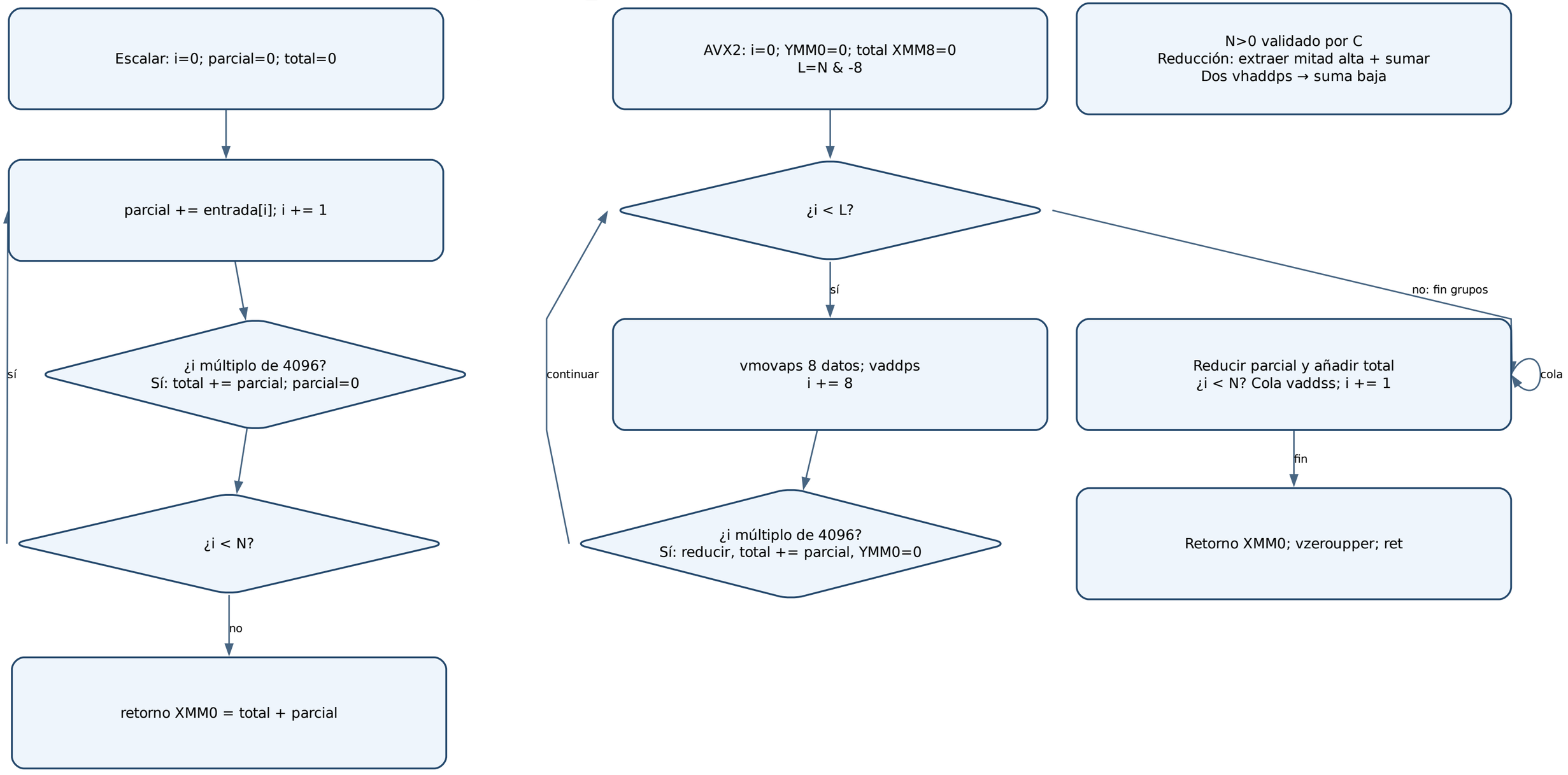
resultados

resultados

Salida: N float32 normalizados
Estadísticas JSON y tiempos por consola

System V AMD64 ABI
sum_array: RDI=entrada, ESI=N; retorno XMM0
compute_stats: RDI, ESI, RDX=&media, RCX=&var, R8=&min, R9=&max
normalize_array: RDI=in, RSI=out, EDX=N, XMM0=media, XMM1=sigma
Registros callee-saved RBX, RBP, R12-R15 y RSP: sin modificación

Flujo de sum_array: escalar y AVX2 con bloques y remanente



compute_stats: dos pasadas, reducción y escrituras ABI

Inicializar índice, parciales y total
Mínimo y máximo = primer elemento válido

escalar

AVX2

Primera pasada escalar
Cargar un float: actualizar suma/min/max
i += 1; cerrar parcial cada 4096
Repetir mientras i<N

Primera pasada AVX2
vmovaps; vaddps; vminps; vmaxps
i += 8; cerrar sumas cada 4096
Reducir suma y extremos; cola escalar

media = suma / float(N)
Reiniciar índice, parcial cuadrados y total

escalar

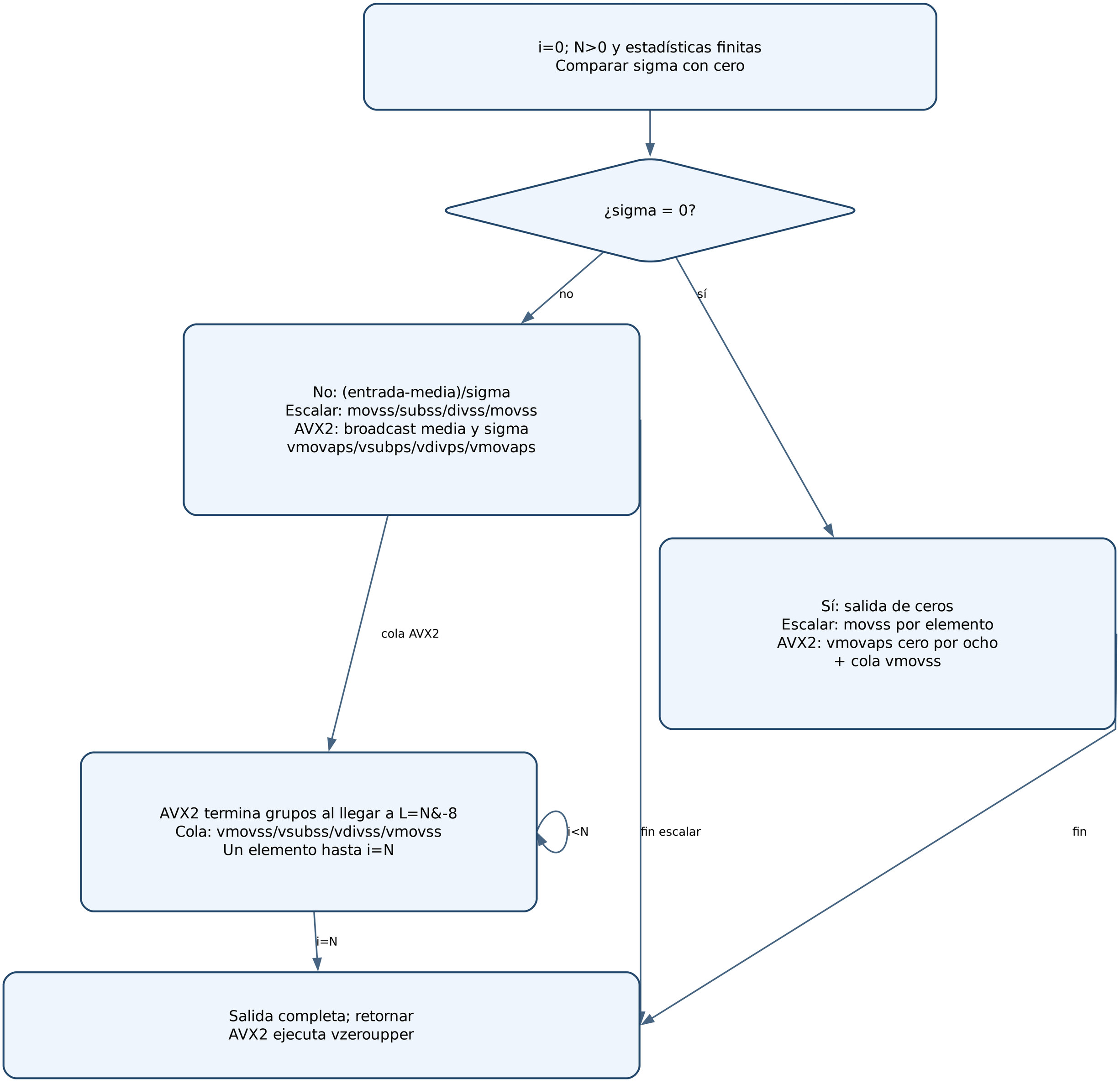
AVX2

Segunda pasada escalar
movss; subss; mulss; addss
Acumular (x-media)^2
i += 1; bloques y último parcial

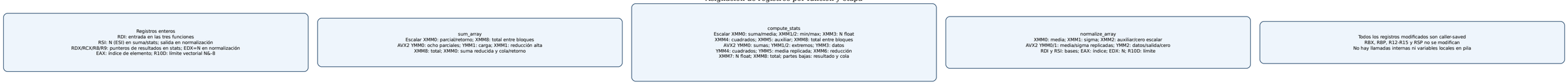
Segunda pasada AVX2
vbroadcastss media; cargar ocho
vsubps; vmulps; vaddps
Bloques, reducción y cola escalar

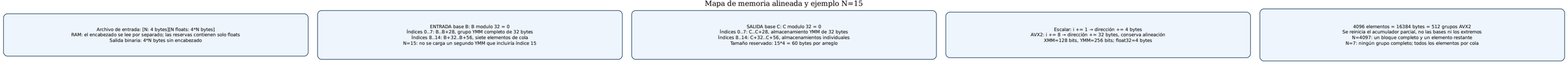
varianza = suma cuadrados / float(N)
Escribir media, var, min, max por RDX/RCX/R8/R9
El driver calcula sigma = sqrtf(varianza)

normalize_array: política sigma=0, grupos completos y cola



Asignación de registros por función y etapa




Mapa de memoria alineada y ejemplo N=15
Archivo de entrada: [N: 4 bytes][N floats: 4*N bytes]
RAM: el encabezado se lee por separado; las reservas contienen solo floats
Salida binaria: 4*N bytes sin encabezado
ENTRADA base B: B modulo 32 = 0
Índices 0..7: B..B+28, grupo YMM completo de 32 bytes
Índices 8..14: B+32..B+56, siete elementos de cola
N=15: no se carga un segundo YMM que incluiría índice 15
SALIDA base C: C modulo 32 = 0
Índices 0..7: C..C+28, almacenamiento YMM de 32 bytes
Índices 8..14: C+32..C+56, almacenamientos individuales
Tamaño reservado: 15*4 = 60 bytes por arreglo
Escalar: i += 1 → dirección += 4 bytes
AVX2: i += 8 → dirección += 32 bytes, conserva alineación
XMM=128 bits, YMM=256 bits; float32=4 bytes
4096 elementos = 16384 bytes = 512 grupos AVX2
Se reinicia el acumulador parcial, no las bases ni los extremos
N=4097: un bloque completo y un elemento restante
N=7: ningún grupo completo; todos los elementos por cola